

Verificação barata no extrator ComprasGov: o experimento HEAD/ETag

Resultado em uma linha: a varredura completa da fonte, que hoje baixa 22,75 GB por rodada para descobrir que nada mudou, passou a custar zero bytes — medido no escopo de produção inteiro (1.212 arquivos, 2021-12 → hoje) e conferido arquivo a arquivo.

29/08/2026 · André Maia (2ª edição; a 1ª, de 28/08, cobria só o recorte) branch `feat/extrator-head-etag` @ `76533d7` (código medido) sobre `main`
`d851bc2` bancada: MinIO local · fonte real `repositorio.dados.gov.br` reproduzível: pacote “reproduzir” anexo (v2)

1 · Por que este teste foi feito

A cada execução, o pipeline `pncp-comprasgov` re-baixa o histórico completo da fonte — 22,75 GB, medidos — só para calcular o hash de cada arquivo e concluir que quase nada mudou. Esse custo motivou a proposta de simplificação em discussão: tornar a checagem opcional e, por padrão, ingerir só arquivos novos.

O problema é que esta fonte reescreve o passado, e em lote: os anuais de 2021, 2022, 2023 e 2024 — todos anos fechados — foram reescritos no mesmo dia, 02/04/2025; o anual de 2025 foi reescrito em 23/12/2025 e 01/01/2026. Nenhum diário ou mensal cobre essas reescritas. “Só arquivos novos” (e também “janela de 30 dias”) perderia quatro anos de uma vez, em silêncio.

Daí a tese alternativa: a checagem não precisa ser opcional — precisa ser barata. Toda URL da fonte responde HEAD com ETag; dá para perguntar “mudou?” sem baixar o corpo. Este experimento tira a tese do papel e a mede — primeiro num recorte (28/08), agora no escopo de produção inteiro.

2 · Como foi feito

O patch. Mudança contida em `ingestion/pncp_comprasgov/extract.py`: o manifesto ganha as colunas `etag` e `last_modified`; quando há ETag conhecido, um HEAD decide se o GET é necessário; qualquer dúvida (sem ETag, HEAD falhou, objeto sumiu do bucket, ETag divergente) cai para o GET de sempre, com conferência de hash. `COLIBRI_VERIFICACAO_COMPLETA=1` desliga o atalho. Acompanham 13 testes unitários da decisão HEAD × GET (os primeiros do repositório) e uma correção de memória necessária para a bancada (ver §4).

EtapA A — recorte, dois braços (28/08). `DATA_INICIO = 2026-07-01` → 186 períodos sondados, 18 arquivos existentes (3,29 GB). Braços *baseline* (código do `main`) e *patch*, buckets próprios, mesmo estado zerado, cinco rodadas. Provou a **equivalência**: statuses, `alteracoes.csv`, manifesto e parquets idênticos entre os braços.

EtapA B — censo HEAD do escopo completo (29/08). Um HEAD por candidato de produção (2021-12-01 → hoje): 5.388 requisições, 54 s, zero bytes de corpo. Como o `main` faz GET incondicional em todo arquivo existente, a soma dos Content-Length é o custo exato de uma rodada do `main` — sem precisar rodá-lo. Cruzamento: para os 18 arquivos do recorte, a soma do censo (3.288.734.846 bytes) é igual, byte a byte, ao que a etapa A mediu por GET.

Etapa C — escopo completo, braço patch (29/08). Três rodadas sobre o escopo inteiro: r1 (primeira carga), r2 (nada mudou) e r3 (sabotagem: ETag adulterado à mão no manifesto do bucket para o anual COMPRA 2022 — um arquivo antigo, do tipo que o recorte não exercitava). Instrumentação por fora, como antes: o driver embrulha a sessão HTTP e o funil de processamento sem alterar o código medido. Veredito automático contra o censo (`comparar_censo.py`).

3 • O que era esperado

- 1. **r1** baixa exatamente os bytes do censo, arquivo a arquivo, e aprende o ETag que o censo viu — a primeira carga não pode ser diferente da de hoje.
- 2. **r2** custa zero bytes: um `HEAD` por arquivo existente, manifesto idêntico ao da r1, nenhuma alteração para o dbt.
- 3. **r3** baixa exatamente um arquivo (o sabotado), confirma por hash que nada mudou e regrava o ETag correto sozinha.

4 • O que conseguimos

RODADA	CENÁRIO	GET C/ CORPO	HEAD	MB BAIXADOS	SEGUNDOS
censo (= <code>main</code> , qualquer rodada)	varredura completa, GET incondicional	1.212	0	22.749,2	—
patch r1	primeira carga, aprende os ETags	1.212	0	22.749,2	3.684
patch r2	nada mudou	0	1.212	0,0	420
patch r3	ETag adulterado no anual COMPRA 2022	1	1.212	40,2	460

Fora da tabela, todas as rodadas fazem as mesmas 4.176 sondagens em períodos inexistentes (GET 404, ~0 bytes). Fonte parada desde 16/07 — cenário “nada mudou” natural. Os 420 s da r2 são 5.388 requisições sequenciais (~78 ms cada); o tempo da r1 inclui a conversão para parquet e o upload de 22,75 GB.

As três hipóteses confirmaram — agora no escopo completo (20 de 20 checagens).

Custo: 22.749,2 MB → 0 MB; 3.684 s → 420 s (8,8×). **Fidelidade:** r1 baixou byte a byte o que o censo previu (1.212/1.212 arquivos) e aprendeu 1.212/1.212 ETags iguais aos do censo; r2 deixou o manifesto idêntico e `alteracoes.csv` vazio. **Robustez:** a sabotagem num anual de ano fechado causou exatamente 1 re-download (40,2 MB), o hash confirmou conteúdo igual e o ETag correto (`"67edc2aa-2660b48"`) foi regravado sem intervenção.

Cobertura. O recorte de 28/08 cobria 1,5 % dos arquivos e 14,5 % dos bytes de produção; a extrapolação de então (~15–20 GB) estava subestimada — são 22,75 GB. O escopo completo agora cobre 100 %.

Achados de bancada que refinam a proposta

- **Memória do extrator (problema pré-existente, independente do HEAD/ETag).** Ao registrar um arquivo no manifesto, o código decodifica o CSV inteiro para `str` e o embrulha num `StringIO` — o que custa $4 \times$ o tamanho do arquivo em RAM (buffer UCS-4), além da `str` e do conteúdo. Para o anual de itens de 2025 (3,86 GB) são ~19 GB extras: a carga não cabe em 15 GB, e o pipeline de produção precisa hoje de mais de 23 GB para esse arquivo. A leitura em streaming (`csv.reader` sobre `TextIOWrapper(BytesIO(...))`) custa +0 MB com a mesma contagem, e foi o que permitiu a rodada completa nesta bancada (pico observado: ~3,5 GB). São 3 linhas, com teste. O próximo limite é o próprio `requests`, que monta o corpo inteiro em RAM — discussão à parte.
- **Duas colunas bastam no manifesto:** o ETag deste nginx já codifica mtime + tamanho. Falso positivo custa um download inofensivo (o hash segura); falso negativo, na prática, não existe.
- **Nada muda fora do `extract.py`:** o `pipeline.py` já sobe o manifesto ao fim de toda rodada — a primeira rodada após o merge aprende os ETags pelo caminho atual (cara, igual à de hoje) e da segunda em diante a varredura fica barata. Manifestos antigos são compatíveis.
- **Sondagens 404:** 4.176 dos 5.388 candidatos de cada rodada são períodos que nunca existiram. Custam ~0 bytes, mas são 77 % das requisições — otimizável, em outra conversa.

5 • Caminhos possíveis a partir daqui

1. **Reproduzir do outro lado.** O pacote “reproduzir” (v2) repete o recorte em ~30–40 min e o escopo completo em ~1h15 (~23 GB de banda; `SPIKE_ROTULO=completo SPIKE_DATA_INICIO=2021-12-01`), em MinIO local ou bucket de teste próprio.
2. **Virar PR.** A branch `feat/extrator-head-etag` já contém o spike, o acabamento (cliente S3 único, falha de HEAD explícita, modo de verificação completa) e os testes; a correção de memória acompanha. Abre quando houver alinhamento de rumo.
3. **Destravar a automação.** Com varredura a custo ~zero, um workflow agendado (após o CI do PR #69) pode rodar o pipeline diariamente sem desperdício — o caminho para a “atualização periódica automática” do MVP.
4. **Generalizar.** O padrão HEAD/ETag pode servir aos demais extratores (NF-e, catmats), conferindo antes o suporte de cada fonte a requests condicionais.
5. **Coordenar com o refactor existente.** A branch `refactor/extrator` (#34) reescreve esta área com classes e testes; o destino dela define se o patch entra no código atual ou na versão refatorada. Decisão de rumo, não técnica.

Evidência completa: censo (`censo-head-2026-08-29.csv`), métricas por request, statuses, manifestos, logs e veredito das rodadas `completo-patch-r1..r3`, mais os resultados do recorte de 28/08, em `spike-head-etag/resultados/` · dossiê “Anatomia da Ingestão ComprasGov”, seção 08. Contexto: conversa sobre simplificação do pipeline (27/08/2026) · fonte parada desde 16/07/2026 · nenhum recurso de produção foi tocado pelo experimento.